

WHEN SHOULD A VISION SPECIALIST DEFER? BUDGETED COLLABORATION WITH VISION-LANGUAGE MODELS

Sihang Cai¹ Rongjunchen Zhang¹ Tao Jin²

¹HiThink Research ²Zhejiang University

ABSTRACT

When should a vision specialist call a general vision-language model? Low specialist confidence identifies uncertain predictions, but does not establish that replacement will help. We study this distinction across four visual tasks and sixteen heterogeneous model pairs. Paired predictions reveal both recoverable errors and correct specialist answers that delegation would damage. We train a compact router to distinguish four paired-outcome states using frozen image/text features and specialist information available before delegation. With thresholds selected on validation and fixed at test time, our router improves on confidence routing in 11 of 16 pairs while reducing calls in 15. Five collaborations outperform both constituent models. At matched call budgets up to 25%, learned ranking yields higher average performance in 15 pairs. These gains depend on model complementarity and reliable threshold transfer, rather than model scale alone. Code, weights and supplementary materials will be available at <https://github.com/USERNAME/REPOSITORY>.

Index Terms— Model collaboration, visual recognition, learning to defer, budgeted inference

1. INTRODUCTION

Vision systems increasingly combine efficient task specialists with general vision-language models (VLMs) that interpret broader visual and linguistic cues. Selective VLM access is attractive for classification, localization and reasoning under limited resources, a demand reflected in commercial prompt-routing services [1]. Once a specialist has answered, which inputs justify a second model call?

However, existing mechanisms only partly address this setting. Speculative decoding verifies draft tokens while preserving the target distribution [2, 3]. This requires a compatible token interface, which detector boxes and classifier logits do not provide. Likewise, language-model routers learn preferences or quality differences within text-generation systems [4, 5]. Our setting instead pairs heterogeneous vision specialists with VLMs: after the specialist runs, replacement is judged by the task’s correctness rule, such as a matching class label or sufficiently overlapping boxes.

SelectiveNet studies rejection [6], learning to defer accounts for imperfect experts [7], and confidence estimation measures a model’s own reliability [8]. Routing and cascading also examine quality–cost trade-offs [9, 10, 11, 12], alongside multimodal studies such as VL-RouterBench [13], MMR-Bench [14], ReLoPe [15] and StreamSense [16]. In contrast, we ask whether replacing an already-computed specialist prediction helps on a fixed visual task. Differences in model interfaces, routing stages and evaluation targets prevent direct comparison of reported scores. Adapting these methods would require reimplementing under the same conditions, so our experiments use confidence routing as a shared baseline.

A confidence threshold defers uncertain inputs. Treating this as escalation assumes that the VLM is better overall and preferable on low-confidence inputs, at the price of another call. Yet neither follows from scale. A stronger VLM can damage correct specialist answers, while a weaker VLM can rescue failures. Even perfect error detection cannot separate recoverable failures from inputs that both models get wrong. Thus, uncertainty alone does not establish recoverability.

We design a compact router to predict paired outcomes before delegation. It combines image/text representations with specialist states, predictions and confidence. Offline predictions supply four training states: both correct, rescue, harm and both wrong. At inference, the router uses only information available before the VLM call to identify recoverable failures and avoid harmful replacements.

Across four tasks and sixteen pairs, InstanceVG/Qwen reaches 75.28% test accuracy with 53.56% calls, exceeding either model alone. However, all four CUB collaborations remain below their specialists. Collaboration is therefore most useful when complementary VLM predictions justify their added cost and validation thresholds transfer reliably. Under strict compute budgets or tight latency constraints, the added VLM call may be unattractive, especially when the specialist is already highly accurate.

Our contributions are threefold. **(1)** We formalize specialist-first collaboration through available rescues, harmful replacements and their selectability, and examine its scope across visual tasks and model pairs. **(2)** We develop a compact four-state router with a common training recipe that improves performance and reduces calls across diverse pairs, sometimes exceeding both models. **(3)** We provide a documented baseline with explicit training, scoring and threshold choices and a reproducible release plan.

2. PRELIMINARY

Collaboration setting. Let S be a specialist, L a VLM, and x the images and query. Their predictions share a task output space. From pre-call information $\mathcal{I}(x)$, a router selects

$$\hat{y}(x) = \begin{cases} S(x), & a(x) = 0, \\ L(x), & a(x) = 1, \end{cases} \quad a(x) \in \{0, 1\}. \quad (1)$$

The action replaces the entire prediction. For metric M and a batch of n inputs, a reference budgeted objective is

$$\max_a M(\hat{\mathbf{y}}, \mathbf{y}) \quad \text{s.t.} \quad \sum_i a(x_i) \leq \lfloor bn \rfloor. \quad (2)$$

The call fraction b is neither a score threshold nor a FLOP budget; each delegated task example counts once. At deployment, a validation-selected threshold acts on each input independently. Actual test calls need not satisfy the batch quota in Eq. 2.

(a) Inputs and vision specialist

(b) Router: five inputs, four states

(c) Budgeted collaboration

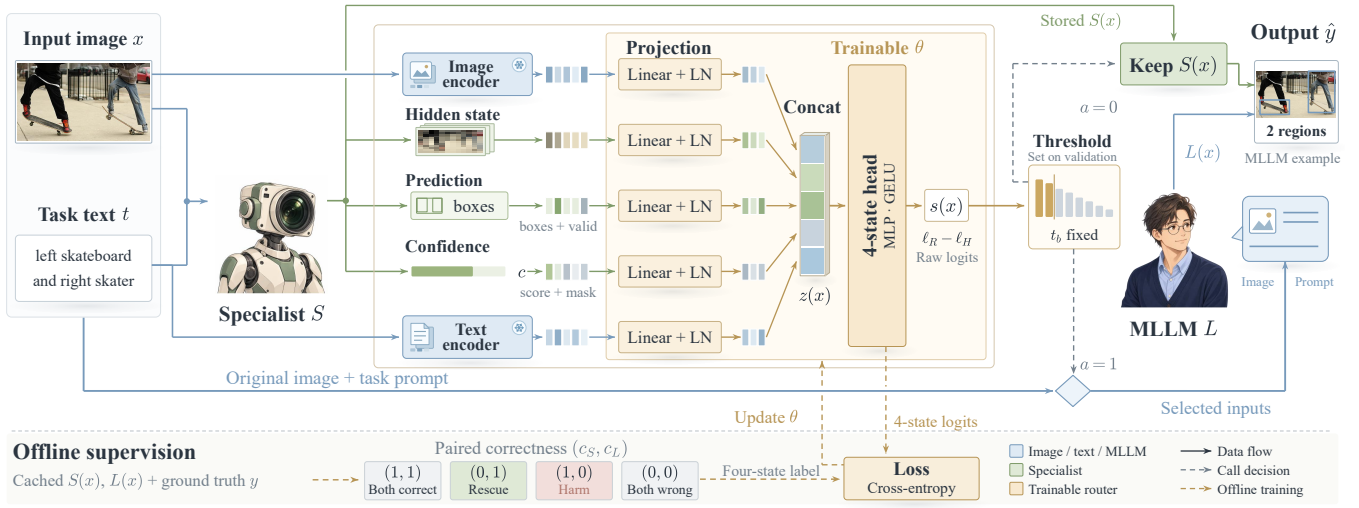


Fig. 1. Deciding when to call the VLM. After the specialist predicts, a compact router combines input and specialist features to estimate four possible outcomes of replacement. Paired model predictions provide training labels. At inference, a threshold chosen on validation data determines whether to keep the specialist answer or call the VLM. Image-only tasks omit text.

3. METHOD

3.1. When Replacement Helps

Let $c_S, c_L \in \{0, 1\}$ indicate specialist and VLM correctness. Their paired outcomes define four states: both correct (1, 1), rescue (0, 1), harm (1, 0) and both wrong (0, 0). Over n examples, the change in accuracy is

$$\Delta \text{Acc} = \frac{|R \cap A| - |H \cap A|}{n}. \quad (3)$$

Here R, H and A are the rescue, harm and delegated sets, and $|\cdot|$ denotes set size. Selecting all rescues and no harms gives the maximum gain $|R|/n$.

Given pre-call information $\mathcal{I}$, expected correctness gain is

$$\eta(\mathcal{I}) = \Pr(R | \mathcal{I}) - \Pr(H | \mathcal{I}). \quad (4)$$

The probabilities refer to rescue and harm for the input. With equal call costs, selecting the largest positive gains maximizes expected accuracy. Error prediction also counts both-wrong cases and can favor useless calls. Macro-F1 is not additive, so we compute it from the combined predictions.

3.2. Four-State Routing

The router uses an 8.07M-parameter image/text backbone distilled from SigLIP2 [17]. The backbone and specialist stay frozen during head training. Distillation and architecture details are in the supplementary materials.

Figure 1 combines image features, pooled specialist features, prediction summaries and confidence with missing-value indicators. Task text adds a branch on gRefCOCO and NLVR2. Each branch is projected to 128 dimensions and layer-normalized, then concatenated into an MLP with 128 hidden units and GELU activation. Each pair learns separate weights with shared training settings.

Paired predictions assign one state per example, or state fractions over four events for ConstructionSite. We train with cross-entropy:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^4 q_{ik} \log p_{ik}, \quad p_i = \text{softmax}(\ell_i). \quad (5)$$

Here N counts training examples, q_{ik} is example i 's observed fraction in state k , and ℓ_i, p_i are its logits and probabilities. We sample uniformly without class weights or label smoothing.

At inference, the routing score compares rescue with harm:

$$s(x) = \ell_R(x) - \ell_H(x) = \log \frac{p_R(x)}{p_H(x)}. \quad (6)$$

Subscripts R, H identify rescue and harm outputs. Larger scores favor delegation. This log probability ratio differs from the expected gain in Eq. 4 and requires no VLM answer or ground truth at inference.

3.3. Model Selection and Inference

We select checkpoints and shared settings by average validation performance over 0–25% calls, the low-budget score:

$$\text{Score}_{25} = \frac{1}{.25} \sum_{k=1}^5 \frac{M(b_{k-1}) + M(b_k)}{2} (b_k - b_{k-1}). \quad (7)$$

Here $b_k = .05k$, $k = 0, \dots, 5$, and $M(b_k)$ is the task metric when the top $\lfloor b_k n \rfloor$ of n validation inputs are delegated. Tied scores retain sample order. This normalized area summarizes low-budget performance. Shared settings average it equally over pairs within tasks, then over tasks.

Table 1. Test results at validation-selected thresholds. Scores and actual call rates are percentages; parentheses give learned-minus-conf. differences in percentage points. **Green** marks improvement and **red** degradation: higher scores and fewer calls are better. Bold compares routed scores; † exceeds both models. M: MiniCPM-V-4.5; Q: Qwen3.8-27B-FP8.

Dataset	Specialist	VLM	spec.	vlm.	conf.	call _{conf}	learned†	call _{learned} ↓
CUB-200-2011	ResNet	M	81.43	26.56	79.93	11.36	81.38 (↑ 1.45)	2.88 (↓ 8.47)
	ResNet	Q	81.43	54.82	81.58	11.36	81.34 (↓ 0.24)	1.43 (↓ 9.92)
	GLSim	M	90.16	26.56	87.69	8.70	89.80 (↑ 2.11)	4.49 (↓ 4.21)
	GLSim	Q	90.16	54.82	88.78	8.70	90.06 (↑ 1.28)	1.66 (↓ 7.04)
gRefCOCO	GroundingDINO	M	26.69	50.01	49.68	98.74	45.62 (↓ 4.06)	43.52 (↓ 55.22)
	GroundingDINO	Q	26.69	74.47	73.87	98.74	66.10 (↓ 7.77)	57.46 (↓ 41.28)
	InstanceVG	M	66.95	50.01	61.88	52.09	66.72 (↑ 4.84)	30.90 (↓ 21.19)
	InstanceVG	Q	66.95	74.47	72.43	68.39	75.28 † (↑ 2.85)	53.56 (↓ 14.83)
NLVR2	BEiT3	M	80.00	90.74	90.77	85.80	90.64 (↓ 0.13)	84.66 (↓ 1.14)
	BEiT3	Q	80.00	90.06	90.24	85.80	90.88 † (↑ 0.64)	76.66 (↓ 9.14)
	ViLT	M	76.42	90.74	90.64	95.93	90.71 (↑ 0.06)	98.97 (↑ 3.04)
	ViLT	Q	76.42	90.06	89.96	95.93	90.00 (↑ 0.04)	94.86 (↓ 1.08)
ConstructionSite	YOLO26x	M	18.13	22.36	25.23	92.54	25.82 † (↑ 0.59)	47.20 (↓ 45.34)
	YOLO26x	Q	18.13	27.00	25.97	92.21	29.88 † (↑ 3.90)	48.77 (↓ 43.44)
	RT-DETR-X	M	7.89	22.36	21.61	97.90	23.00 † (↑ 1.39)	78.83 (↓ 19.07)
	RT-DETR-X	Q	7.89	27.00	26.82	98.60	26.69 (↓ 0.13)	83.12 (↓ 15.48)

We choose a threshold on validation data over 1–99% call budgets. At budget b , threshold t_b includes tied scores only if all fit the budget. The highest task score wins, with ties favoring fewer calls and then a smaller budget.

The threshold and tie rule apply unchanged to each test input. Score shifts can therefore change actual calls. Confidence follows the same selection procedure with negative maximum class probability or detection confidence, sending missing-confidence inputs first. Test labels are used only for evaluation.

The search excludes never-call and always-call choices, so its result can be worse than the specialist alone. Equal-call comparisons separate ranking quality from threshold transfer.

4. EXPERIMENTS

4.1. Datasets and Metrics

Table 2. Datasets, tasks and dataset split sizes.

Dataset	Task	Train	Val.	Test
CUB-200-2011	Fine-grained classification	4,792	1,200	5,794
gRefCOCO	Semantic grounding	209,344	14,229	35,263
NLVR2	Visual reasoning	85,042	6,982	13,937
ConstructionSite	Event detection	6,308	701	3,004

CUB-200-2011 (CUB) [18] tests fine-grained bird recognition, while NLVR2 [19] tests statements about image pairs. Both use accuracy. Table 2 counts images for CUB and ConstructionSite, expressions for gRefCOCO, and image-pair/statement examples for NLVR2.

gRefCOCO [20] evaluates whole-expression box correctness: after score-.7 filtering and greedy matching at generalized IoU $\geq .5$, no instance may be missed or added. No-target expressions require valid empty outputs. This differs from official segmentation evaluation.

ConstructionSite [21] uses macro-F1 across personal-protection, fall-protection, unprotected-edge and excavator-proximity violations. Each image receives one routing decision for all four events.

4.2. Implementation and Baselines

Specialists are ResNet-50 [22], GLSim [23], GroundingDINO [24], InstanceVG [25], BEiT3 [26], ViLT [27], YOLO26x [28] and RT-DETR-X [29]. Each pairs with MiniCPM-V-4.5 [30] (M) and Qwen3.8-27B-FP8 [31] (Q). Table 1 compares both standalone models and routers.

Router training uses AdamW with learning rate .003, weight decay .01, batch 256, up to 50 epochs and patience 8, with specialist states at 90% depth. Eighteen settings require 288 fits; final main-table fits use seed 42. **All experiments use validation data for parameter selection and test data for evaluation.**

VLMs use one temperature-zero request per example; invalid responses are recorded and never repaired. CUB ResNet’s validation exposure and prior test inspection preclude a newly blinded evaluation. **Supplementary dataset details, experimental settings, full Validation/Test results, additional ablations and visual analyses are available in the code repository.**

4.3. Main Results and Applicability

Learned routing improves on confidence for **11/16** pairs and reduces calls for **15/16**; five pairs exceed both standalone models (Table 1). InstanceVG/Q achieves 75.28% accuracy with 53.56% calls, while YOLO26x/Q gains 3.90 macro-F1 points over confidence with roughly half as many calls. Gains are not universal: CUB collaborations remain below their specialists, and GroundingDINO/Q suffers from threshold transfer.

Applicability boundaries of model collaboration. Collaboration is most promising when the VLM substantially outperforms the specialist, offers recoverable errors, and the application can tolerate its added computation. Aggregate VLM superiority alone is insufficient: gains require distinguishing rescues from harmful replacements and transferring thresholds reliably. Each VLM call can be far more expensive than specialist inference, so even selective delegation may be unsuitable under strict compute or latency limits. Collaboration should therefore be justified by the pair’s complementarity and cost–performance trade-off, rather than applied indiscriminately.

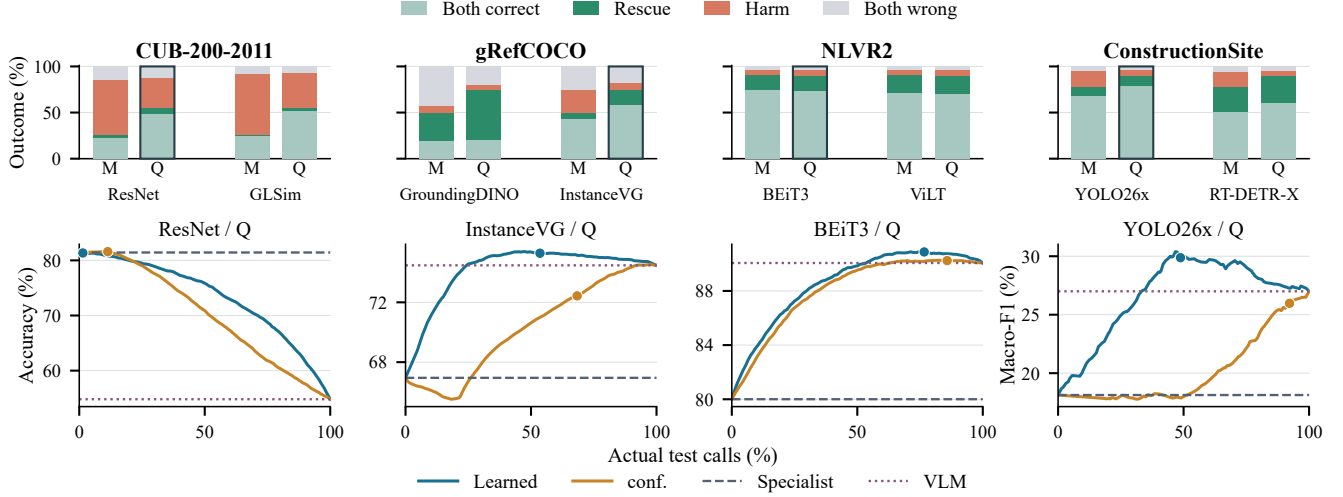


Fig. 2. Where collaboration gains come from (test data). Top: four-state shares for all sixteen pairs; outlines identify the pairs below. Construction counts events; other tasks count examples. Bottom: performance versus actual test calls for 99 validation-selected thresholds and the two endpoints. Dots mark validation-selected policies; lines connect saved points without smoothing.

4.4. Where Collaboration Gains Come From

Figure 2 shows how available complementarity becomes a gain. For InstanceVG/Q, 5,423 test expressions are rescuable and 2,771 are potentially harmful. Selecting 4,361 rescues and 1,423 harms adds 2,938 correct answers, an 8.33-point gain over the specialist. Avoiding 1,348 harms while missing 1,062 rescues also yields 286 more correct answers than Q. CUB has only 106–365 rescuable images versus 1,907–3,791 potentially harmed predictions. These ranges are the minimum and maximum counts across its four model pairs. Both-wrong calls provide no accuracy gain. Construction macro-F1 is evaluated separately from event counts.

Figure 2 uses actual test calls. We also rank each test batch by frozen scores and delegate exactly $\lfloor bn_{\text{test}} \rfloor$ inputs, without labels determining their order. Learned ranking improves test Score₂₅ on **15/16** pairs; ResNet/Q is the exception. Low-budget gains are 1.873–8.028 points on gRefCOCO but only .100–.640 on NLVR2 (100Score₂₅). Better low-budget ranking can coexist with similar selected maximum-performance scores.

4.5. Ablation Experiments

Table 3. Supervision objectives (sample means minimized).

Target	Objective	100Score ₂₅
Error prediction	$\text{BCE}(\hat{e}, e)$	65.294
Gain regression	$(\hat{g} - (y_R - y_H))^2$	66.528
Four-state	$-\sum_k y_k \log p_k$	66.787

Table 3 holds other final settings fixed. Here e denotes specialist error and y_R, y_H rescue and harm targets, averaged over events for ConstructionSite. Error prediction uses binary cross-entropy (BCE), treating rescuable and both-wrong cases alike. Gain regression fits net replacement benefit with squared error. Four-state supervision separates these outcomes and leads task-balanced validation 100Score₂₅ by .259 points over regression. This modest gain supports explicit rescue–harm modeling, without establishing the necessity of every architectural choice.

4.6. Computational Cost

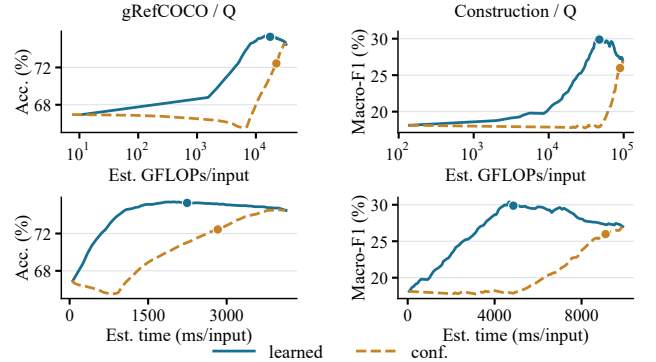


Fig. 3. Cost–performance trade-offs. InstanceVG/Qwen (left), YOLO26x/Qwen (right). Test performance versus estimated cost; dots mark the Table 1 policies selected on validation.

On A800, each component is timed on 64 inputs at batch one. Learned routing includes the distilled encoders and routing head. We estimate per-input FLOPs and serial inference time by combining measured component costs with actual test call rates.

With Qwen on both tasks, learned routing gains 2.85 and 3.90 points over confidence while reducing estimated FLOPs by 21.7% and 47.0%, and time by 20.7% and 46.6%, respectively. Better selection therefore improves performance at lower cost than confidence routing, although VLM calls still add substantial cost relative to specialist-only inference.

5. CONCLUSION

Four-state routing improves on confidence across most pairs, often with fewer calls and sometimes beyond both models. Cost estimates also favor learned selection. These gains depend on complementary models, reliable threshold transfer and available resources, making collaboration a task-specific choice.

6. REFERENCES

- [1] Amazon Web Services, “Understanding intelligent prompt routing in Amazon Bedrock,” Product documentation, Accessed September 10, 2026.
- [2] Yaniv Leviathan, Matan Kalman, and Yossi Matias, “Fast inference from transformers via speculative decoding,” in *Proceedings of the 40th International Conference on Machine Learning*, 2023, vol. 202 of *Proceedings of Machine Learning Research*, pp. 19274–19286, PMLR.
- [3] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper, “Accelerating large language model decoding with speculative sampling,” *arXiv preprint arXiv:2302.01318*, 2023.
- [4] Lingjiao Chen, Matei Zaharia, and James Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” *arXiv preprint arXiv:2305.05176*, 2023.
- [5] Isaac Ong, Amjad Almahairi, Vincent Wu, et al., “RouteLLM: Learning to route LLMs with preference data,” *arXiv preprint arXiv:2406.18665*, 2024.
- [6] Yonatan Geifman and Ran El-Yaniv, “SelectiveNet: A deep neural network with an integrated reject option,” in *Proc. ICML*, 2019, pp. 2151–2159.
- [7] Hussein Mozannar and David Sontag, “Consistent estimators for learning to defer to an expert,” in *Proc. ICML*, 2020, pp. 7076–7087.
- [8] Chuan Guo, Geoff Pleiss, Yu Sun, et al., “On calibration of modern neural networks,” in *Proc. ICML*, 2017, pp. 1321–1330.
- [9] Jasper Dekoninck, Maximilian Baader, and Martin Vechev, “A unified approach to routing and cascading for LLMs,” *arXiv preprint arXiv:2410.10347*, 2024.
- [10] Xinyuan Wang, Yanchi Liu, Wei Cheng, Xujiang Zhao, Zhengzhang Chen, Wenchao Yu, Yanjie Fu, and Haifeng Chen, “MixLLM: Dynamic routing in mixed large language models,” in *Proc. NAACL-HLT*, Apr. 2025, pp. 10912–10922.
- [11] Ruihan Jin, Pengpeng Shao, Zhengqi Wen, Jinyang Wu, Mingkuan Feng, Shuai Zhang, and Jianhua Tao, “Radial-Router: Structured representation for efficient and robust large language models routing,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*, Nov. 2025, pp. 14587–14600.
- [12] Clovis Varangot-Reille, Christophe Bouvard, Mathieu Ciancone, et al., “Doing More with Less: A Survey on Routing Strategies for Resource Optimisation in Large Language Model-Based Systems,” *Journal of Artificial Intelligence Research*, vol. 86, 2026.
- [13] Zhehao Huang, Baijiong Lin, Jingyuan Zhang, Jingying Wang, Yuhang Liu, Ning Lu, Tao Li, and Xiaolin Huang, “VL-RouterBench: A benchmark for vision–language model routing,” *arXiv preprint arXiv:2512.23562*, 2025.
- [14] Haoxuan Ma, Guannan Lai, and Han-Jia Ye, “MMR-Bench: A comprehensive benchmark for multimodal LLM routing,” *arXiv preprint arXiv:2601.17814*, 2026.
- [15] Yaopei Zeng, Congchao Wang, Blake JianHeng Chen, and Lu Lin, “ReLope: KL-regularized LoRA probes for multimodal LLM routing,” *arXiv preprint arXiv:2603.24787*, 2026.
- [16] Han Wang, Deyi Ji, Lanyun Zhu, Jiebo Luo, and Roy Ka-Wei Lee, “StreamSense: Streaming social task detection with selective vision–language model routing,” in *Proceedings of the ACM Web Conference 2026*, 2026.
- [17] Michael Tschanen, Alexey Gritsenko, Xiao Wang, et al., “SigLIP 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features,” *arXiv preprint arXiv:2502.14786*, 2025.
- [18] C. Wah, S. Branson, P. Welinder, et al., “The Caltech-UCSD Birds-200-2011 dataset,” Tech. Rep. CNS-TR-2011-001, California Institute of Technology, 2011.
- [19] Alane Suhr, Stephanie Zhou, Ally Zhang, et al., “A corpus for reasoning about natural language grounded in photographs,” in *Proc. Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 6418–6428.
- [20] Chang Liu, Henghui Ding, and Xudong Jiang, “GRES: Generalized referring expression segmentation,” in *Proc. IEEE/CVF CVPR*, 2023, pp. 23592–23601.
- [21] Xuezheng Chen and Zhengbo Zou, “Are large pre-trained vision language models effective construction safety inspectors,” *Data-Centric Engineering*, vol. 7, pp. e11, 2026.
- [22] Kaiming He, Xiangyu Zhang, Shaoqing Ren, et al., “Deep residual learning for image recognition,” in *Proc. IEEE/CVF CVPR*, 2016, pp. 770–778.
- [23] Edwin Arkel Rios, Min-Chun Hu, and Bo-Cheng Lai, “Global-Local Similarity for Efficient Fine-Grained Image Recognition with Vision Transformers,” in *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2025, pp. 1–5.
- [24] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” in *Proc. European Conference on Computer Vision*, 2024, pp. 38–55.
- [25] Ming Dai, Wenxuan Cheng, Jiang-Jiang Liu, et al., “Improving Generalized Visual Grounding With Instance-Aware Joint Learning,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 48, pp. 448–465, 2026.
- [26] Wenhui Wang, Hangbo Bao, Li Dong, et al., “Image as a foreign language: BEiT pretraining for vision and vision-language tasks,” in *Proc. IEEE/CVF CVPR*, 2023, pp. 19175–19186.
- [27] Wonjae Kim, Bokyoung Son, and Ildoo Kim, “ViLT: Vision-and-language transformer without convolution or region supervision,” in *Proc. ICML*, 2021, pp. 5583–5594.
- [28] Glenn Jocher, Jing Qiu, Mengyu Liu, et al., “Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models,” *arXiv preprint arXiv:2606.03748*, 2026.
- [29] Yian Zhao, Wenyu Lv, Shangliang Xu, et al., “DETRs Beat YOLOs on Real-time Object Detection,” in *Proc. IEEE/CVF CVPR*, 2024, pp. 16965–16974.
- [30] Tianyu Yu, Zefan Wang, Chongyi Wang, et al., “MiniCPM-V 4.5: Cooking Efficient MLLMs via Architecture, Data, and Training Recipe,” *arXiv preprint arXiv:2509.18154*, 2025.
- [31] Qwen Team, “Qwen3.8-27B-FP8,” Official model card, 2026.